

Bank Account Management System (BAMS)

Consolidated Low-Level Design (LLD) — Implementation Contract for AI-Assisted Development

BAMS Engineering Team
September 2026

Contents

1 Document Purpose & Consolidation Note	3
2 AI Coding Execution Contract	3
3 Software Development Life Cycle (SDLC) Model	4
3.1 Model Selected: Waterfall	4
3.2 Why Waterfall Is Suitable Here	4
3.3 Waterfall Phases Mapped to This Document	5
4 Technology Stack	5
5 Architecture	6
5.1 Style	6
5.2 Request Flow	6
5.3 Module Boundaries	7
6 Solution / Project Structure	7
7 Complete Frontend Specification	8
7.1 Full Page List by Role	8
7.2 Route-to-Role Visibility	9
8 Domain Entities (Exact Fields)	10
8.1 Account	10
8.2 Transaction	11
8.3 ApplicationUser / Identity Core	11
8.4 Domain Enumerations	11
9 Database Design — Indexes & Constraints (SQL Server)	12
10 RBAC — Roles, Permissions & Enforcement	13
10.1 Roles	13
10.2 Seed Permissions by Role	13
10.3 Policy Registration (Program.cs)	13
10.4 Full Authorization Matrix	14
11 Frontend ↔ Backend Permission Mapping	14
12 Database Seed Data	15
12.1 Roles	15
12.2 Default Administrator Account	15
12.3 Reference Data	15
13 Authentication — JWT Contract	16
13.1 Claims in the Access Token	16
13.2 Endpoints	16

13.3 Password Policy & Lockout	16
13.4 Middleware Pipeline Order (Program.cs)	17
14 Configuration & Environment Variables	17
14.1 Backend (appsettings.json)	17
14.2 Frontend (Vite Environment Variables)	17
14.3 Configuration Files Required	18
15 API Contract (Representative Endpoints)	18
15.1 Authentication	18
15.2 Bank Accounts	18
15.3 Transactions	19
15.4 System Save & Restore	19
15.5 Error Responses (All Endpoints)	19
16 API Request & Response Examples	20
17 Repository / Service Interfaces & DTOs	22
17.1 Service Contracts	22
17.2 Example Service Method (TransferAsync: ACID + Concurrency + Logging)	22
17.3 ITransactionService	23
17.4 Key DTOs	23
18 Complete DTO Specification	24
19 Financial & Transaction Engine — Deep Detail	25
19.1 Balance & Transaction Formulas	25
19.2 Transfer Processing Flow	26
19.3 Concurrency Control & RowVersion Mechanics	26
20 Structured Logging & Audit Trail	27
20.1 Serilog Sinks	27
20.2 Canonical Event Names	27
20.3 Never Logged	27
20.4 Correlation ID Tracing	27
21 SOLID Principles — Implementation Mapping	28
22 Design Patterns — Implementation Mapping	29
22.1 OOP Principles Applied	29
23 Security, Exception Handling & Production Hardening	30
23.1 Exception Handling Contract	30
23.2 Security Design	30
23.3 Security Testing Checklist	31
23.4 Deployment	31
23.5 Exact Configuration / Launch Profile	31
23.6 Git / Branch / Commit Rules	32
24 Testing Matrix	32
25 Sample Test Case Table	33
26 Required Diagrams	34
27 Final Acceptance Criteria	34
28 Definition of Done (Per Module)	35
29 Build Order & Working Agreement for an AI Coding Tool	35
29.1 Build Order (Waterfall-Sequenced)	35
29.2 Working Agreement	36

30 One-Minute Team Explanation 36

31 AI Project Generation Prompt 36

1 Document Purpose & Consolidation Note

This is a single, self-contained Low-Level Design (LLD) document for the Bank Account Management System (BAMS). It merges the specifications across Phase 1 (Authentication & User Management), Phase 2 (Account Management & Ownership), Phase 3 (Financial Transactions & Concurrency), Phase 4 (Transaction History & Querying), Phase 5 (System Save & Restore / Backup & Rehydration), and Phase 6 (Production Hardening) into one coherent engineering contract, so it can be handed to an AI coding tool (or an engineer) with the instruction “*implement exactly this*” and produce a consistent, working, end-to-end codebase.

Two consolidation decisions were made while merging the source specifications:

1. **Single technology stack.** The system standardizes on **ASP.NET Core Web API (C# / .NET 8) + React 18 (Vite) + Microsoft SQL Server + Entity Framework Core 8**. All database DDL, types, and constraints are written in SQL Server dialect (`decimal(18,2)` , `rowversion` , `datetime2` , `nvarchar`).
2. **Implemented items only.** Technologies and architectural patterns flagged as optional or unselected (e.g., Redis distributed caching, SignalR real-time hubs, Razor Pages, microservices, containerization) are omitted or categorized as not implemented to ensure an unambiguous contract for automated development.

This document specifies the AI Coding Execution Contract, a complete frontend page/route specification, field-by-field DTO contracts, database seed data, environment configurations, a Definition of Done checklist, literal request/response examples, frontend-to-backend permission mappings, testing matrices, and a ready-to-copy AI project generation prompt.

2 AI Coding Execution Contract

This section governs how any AI coding tool (or developer) must behave while building the BAMS from this document. It is binding for the entire build:

1. **Treat this document as the single source of truth.** If something is not specified here, stop and flag it rather than inventing a convention silently.
2. **Do not change technology, architecture, entities, API contracts, roles, permissions, business rules, or database design without explicit approval.**
3. **Do not generate placeholder implementations.** No `TODO` , no `NotImplementedException` , no empty method bodies, no fake in-memory repositories standing in for real ones, no hard-coded mock API responses.
4. **Build the project incrementally, following the Build Order (§29.1)** — one phase at a time, never the whole application in one pass.
5. **After completing each phase:**
 - restore/install dependencies (backend and frontend)
 - compile the backend with 0 errors and 0 warnings
 - build the frontend
 - run the unit and integration tests for that phase
 - verify database migrations apply cleanly against SQL Server
6. **Do not proceed to the next phase if the current phase does not build or does not pass its tests.**
7. **All backend API responses must use standard DTOs or structured envelopes** — no endpoint returns a bare internal entity or exposes passwords/hashes.
8. **Never expose EF Core entities directly through controllers** — every request/response crosses the API boundary as a DTO (§17.4, §18).
9. **Never bypass authorization** — every endpoint carries the policy listed for it; no endpoint is left `[AllowAnonymous]` unless explicitly specified (only `/api/v1/auth/register` and `/api/v1/auth/login`).
10. **Never put business logic inside controllers** — controllers route, validate models, and invoke services; balance calculations, account type constraints, and transfer validations live in the Service Layer.

11. **Never put database access directly inside controllers or React components** — controllers depend on service interfaces; React components call centralized API client functions.
12. **Use dependency injection according to the architecture** — all services are constructor-injected and registered with `Scoped` lifetime in `Program.cs`.
13. **Maintain optimistic concurrency using EF Core RowVersion** on accounts to prevent balance race conditions.
14. **Generate all required tests listed in the Testing Matrix (§24)** — a module is not done without verification.
15. **At completion, verify every item in the Final Acceptance Criteria (§27)** before declaring the project finished.

3 Software Development Life Cycle (SDLC) Model

3.1 Model Selected: Waterfall

The BAMS project follows the Waterfall SDLC model. Requirements and architecture are completely determined up-front to deliver a secure financial platform:

Requirements -> System Design (this LLD) -> Implementation -> Integration & Testing -> Deployment -> Maintenance

3.2 Why Waterfall Is Suitable Here

- **Requirements are fully known and fixed up front.** In banking, ledger accounting rules, cash-only savings constraints, and optimistic concurrency cannot drift mid-development.
- **Linear dependency-ordered build path.** Identity & Auth must exist before Accounts; Accounts must exist before Transactions; Transactions must exist before Querying and Backup/Restore.
- **AI-Assisted single-pass execution.** AI tools perform best against an unambiguous, fully specified contract without renegotiating requirements.
- **Strict acceptance gates.** Financial software requires comprehensive verification against fixed criteria before production sign-off.

3.3 Waterfall Phases Mapped to This Document

Phase	Waterfall Activity	Section of This Document
1. Requirements	Banking business rules, Savings/Checking rules, RBAC	§5, §8, §19
2. System Design	Architecture, schema, API contracts, DTOs, Concurrency	§4, §6, §7, §9, §10, §13, §15, §18
3. Implementation	Phased build order, service logic, controllers, React SPA	§17, §19, §29
4. Integration & Testing	Unit, integration, and security verification	§24, §25
5. Deployment & Hardening	Serilog rolling file logs, Correlation IDs, Health checks	§20, §23
6. Maintenance	System save/restore, backup verification, audit trail	§15.4, §20.4

4 Technology Stack

Layer	Technology	Usage
Frontend	React 18 + Vite 5, React Router v6, Axios, CSS Modules	Single Page Application (SPA), protected routes, auth interceptor
Backend API	ASP.NET Core Web API (.NET 8.0 LTS, C# 12)	RESTful API controllers, custom middleware pipeline, DI
ORM / Data Access	Entity Framework Core 8.0.10 (Code-First)	LINQ queries, transactions, optimistic concurrency, migrations
Database	Microsoft SQL Server (Express / Developer)	Relational store, unique indexes, rowversion concurrency token
Authentication	ASP.NET Core Identity + JWT Bearer (HMAC-SHA256)	User credential storage, PBKDF2 hashing, signed access tokens
Authorization	Role-Based + Resource Ownership Validation	Admin and User roles, account ownership verification
Validation	Data Annotations + [ApiController] Automatic 400	Declarative request model validation, Range, Required
Logging	Serilog (Console + Daily Rolling File Sink)	Structured logging to Logs/bams-.log with 30-day retention
Rate Limiting	ASP.NET Core RateLimiter (Fixed Window)	10 requests/min per IP partition on /api/v1/auth/login
Health Checks	Microsoft.AspNetCore.Diagnostics.HealthChecks	GET /health monitoring API & SQL Server connectivity
API Documentation	Swagger / OpenAPI (Swashbuckle.AspNetCore)	Interactive API exploration and JWT Bearer authorization UI

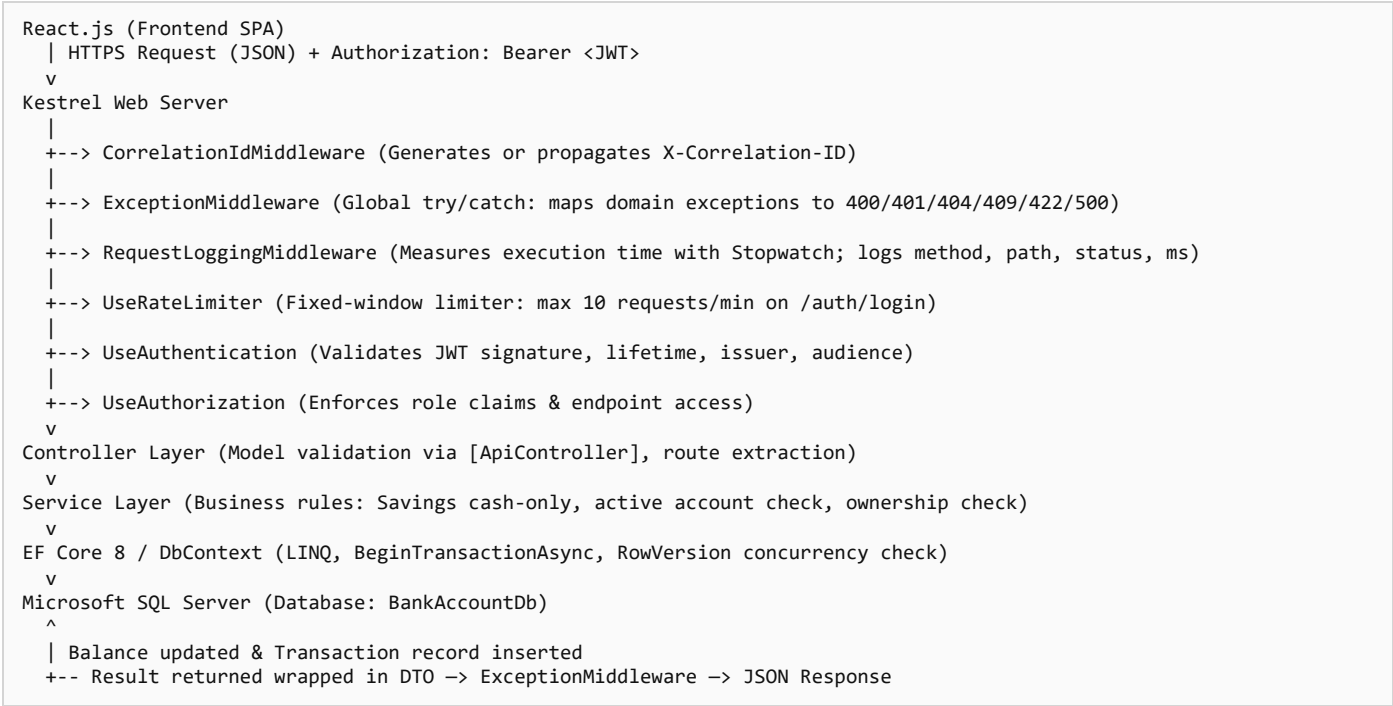
5 Architecture

5.1 Style

Clean Layered (N-Tier) Architecture deployed as a modular Web API backend paired with a decoupled React SPA client:

Layer	Responsibility	Concrete Examples
API Layer	HTTP transport, route dispatching, model validation, Swagger	AuthController, AccountsController, TransactionsController, SystemController
Middleware Layer	Cross-cutting pipeline concerns, tracing, exceptions, telemetry	CorrelationIdMiddleware, ExceptionMiddleware, RequestLoggingMiddleware
Service Layer	Business rules, ACID transfer orchestration, concurrency checks	AuthService, AccountService, TransactionService, SystemBackupService
Data Layer	Entity modeling, database context, migration history	ApplicationDbContext, Account, Transaction, ApplicationUser, Migrations
Client Layer	User interface, client-side routing, JWT storage, state context	React components, AuthContext, Axios instance, CSS Modules

5.2 Request Flow



5.3 Module Boundaries

The system enforces modular boundaries across four primary business domains:

- Authentication & User Management:** Manages `ApplicationUser`, Identity roles (`Admin`, `User`), registration, login, and JWT generation.
- Account Management:** Manages bank accounts, account number formatting (`ACC-YYYYMMDD-XXXX`), ownership validation, and soft deactivation (`IsActive = false`).
- Transaction Management:** Executes deposits, withdrawals, and atomic transfers. Enforces Savings cash-only constraint, Checking cheque support, balance sufficiency, and optimistic concurrency checks.

- **System Backup & Rehydration:** Admin-only module providing full database export to JSON and atomic transactional rehydration.

6 Solution / Project Structure

```

BankAccountManagementSystem/
├── Backend/
│   ├── BankAccountManagementSystem.API/
│   │   ├── BankAccountManagementSystem.API/
│   │   │   ├── Controllers/
│   │   │   │   ├── AccountsController.cs          # /api/v1/accounts
│   │   │   │   ├── AuthController.cs             # /api/v1/auth
│   │   │   │   ├── HomeController.cs             # MVC default template controller
│   │   │   │   ├── SystemController.cs           # /api/v1/system (save & restore)
│   │   │   │   ├── TransactionsController.cs      # /api/v1/transactions
│   │   │   │   ├── UsersController.cs           # /api/v1/users (Admin CRUD)
│   │   │   ├── Services/
│   │   │   │   ├── Interfaces/
│   │   │   │   │   ├── IAccountService.cs
│   │   │   │   │   ├── IAuthService.cs
│   │   │   │   │   ├── ISystemBackupService.cs
│   │   │   │   │   ├── ITransactionService.cs
│   │   │   │   │   ├── IUserService.cs
│   │   │   │   ├── Implementations/
│   │   │   │   │   ├── AccountService.cs
│   │   │   │   │   ├── AuthService.cs
│   │   │   │   │   ├── SystemBackupService.cs
│   │   │   │   │   ├── TransactionService.cs
│   │   │   │   │   └── UserService.cs
│   │   │   ├── Models/
│   │   │   │   ├── Account.cs                    # Balance, AccountType, RowVersion
│   │   │   │   ├── AccountType.cs                # Enum: Savings (0), Checking (1)
│   │   │   │   ├── ApplicationUser.cs            # Extends IdentityUser
│   │   │   │   ├── Transaction.cs                # Amount, Type, Mode, Status, RefNumber
│   │   │   │   ├── TransactionMode.cs            # Enum: Cash (0), Cheque (1)
│   │   │   │   ├── TransactionStatus.cs          # Enum: Pending (0), Completed (1), Failed (2)
│   │   │   │   └── TransactionType.cs            # Enum: Deposit (0), Withdrawal (1), Transfer (2)
│   │   │   ├── DTOs/
│   │   │   │   ├── Accounts/                     # CreateAccountRequest
│   │   │   │   ├── Auth/                        # LoginRequest, LoginResponse, RegisterRequest
│   │   │   │   ├── Common/                      # PagedResult<T>
│   │   │   │   ├── System/                      # SystemBackupDto, RestoreResponseDto
│   │   │   │   ├── Transactions/                 # DepositRequest, WithdrawalRequest, TransferRequest, Query
│   │   │   │   └── Users/                      # CreateUserRequest, UserResponse, AssignRoleRequest
│   │   │   ├── Middleware/
│   │   │   │   ├── CorrelationIdMiddleware.cs    # X-Correlation-ID injection & extraction
│   │   │   │   ├── ExceptionMiddleware.cs        # Centralized 400/401/404/409/422/500 handler
│   │   │   │   └── RequestLoggingMiddleware.cs    # Stopwatch HTTP timing & status logger
│   │   │   ├── Data/
│   │   │   │   ├── ApplicationDbContext.cs       # EF Core IdentityDbContext, Fluent API
│   │   │   │   └── DbInitializer.cs              # Seeds Admin/User roles & default admin
│   │   │   ├── Configuration/
│   │   │   │   └── JwtSettings.cs                # Strongly typed JWT options
│   │   │   ├── Exceptions/
│   │   │   │   └── BankingExceptions.cs          # InsufficientBalance, AccountInactive, InvalidAccountType
│   │   │   ├── Migrations/
│   │   │   │   └── 4 Code-First migrations
│   │   │   ├── Logs/
│   │   │   │   └── bams-.log rolling daily files
│   │   │   ├── Program.cs
│   │   │   └── appsettings.json                  # WebApplicationBuilder, DI, Pipeline configuration
│   │   │   └── Connection strings, JWT settings, Serilog
│   └── Frontend/
│       ├── bank-management-frontend/
│       │   ├── src/
│       │   │   ├── api.js                       # Axios instance, Bearer token interceptor, 401 redirect
│       │   │   ├── App.jsx                      # BrowserRouter & Route configuration
│       │   │   ├── context/
│       │   │   │   └── AuthContext.jsx          # User, Token, Role, Login, Logout state
│       │   │   ├── components/
│       │   │   │   ├── BankingNavigation.jsx    # Sidebar/Navbar links
│       │   │   │   └── ProtectedRoute.jsx       # Token & Role authentication guard
│       │   │   ├── layouts/
│       │   │   │   └── DashboardLayout.jsx      # Shell with header, navigation, and page outlet
│       │   │   ├── pages/
│       │   │   │   ├── LoginPage.jsx           # /login
│       │   │   │   ├── RegisterPage.jsx        # /register
│       │   │   │   └── DashboardPage.jsx        # /dashboard

```

```
├── AccountsPage.jsx          # /accounts
├── AddAccountPage.jsx        # /accounts/new
├── AccountDetailsPage.jsx    # /accounts/:id
├── DepositPage.jsx           # /transactions/deposit
├── WithdrawPage.jsx          # /transactions/withdraw
├── TransferPage.jsx          # /transactions/transfer
├── TransactionHistoryPage.jsx # /transactions/history
├── BackupRestorePage.jsx     # /system/backup-restore (Admin only)
├── ProfilePage.jsx           # /profile
├── services/
│   ├── accountService.js     # Calls /api/v1/accounts
│   ├── transactionService.js  # Calls /api/v1/transactions
│   ├── systemService.js      # Calls /api/v1/system
│   └── userService.js        # Calls /api/v1/users
└── package.json
```

7 Complete Frontend Specification

7.1 Full Page List by Role

Public (Unauthenticated):

- `/login` — User/Admin authentication page
- `/register` — Customer registration page

Authenticated User:

- `/dashboard` — Overview of personal accounts, recent transactions, and quick action cards
- `/accounts` — List of owned accounts, balance summaries, active/inactive badges
- `/accounts/new` — Form to open a new Savings or Checking account
- `/accounts/:id` — Detailed account view with associated transaction history
- `/transactions/deposit` — Deposit funds (Cash or Cheque; Cheque disabled for Savings)
- `/transactions/withdraw` — Withdraw funds (Cash or Cheque; Cheque disabled for Savings)
- `/transactions/transfer` — Transfer funds between Checking accounts
- `/transactions/history` — Searchable, paginated transaction ledger
- `/profile` — Authenticated user details, employee code, assigned roles

Administrator (Admin Role):

- All User views listed above
- Global account visibility across all users in `/accounts`
- `/system/backup-restore` — Export complete database state to JSON and restore from backup file
- Global transaction inspection across all bank accounts

7.2 Route-to-Role Visibility

Route	Page Component	Access Requirement
<code>/login</code>	LoginPage.jsx	Public (Unauthenticated)
<code>/register</code>	RegisterPage.jsx	Public (Unauthenticated)
<code>/dashboard</code>	DashboardPage.jsx	Authenticated (User, Admin)
<code>/accounts</code>	AccountsPage.jsx	Authenticated (User sees own; Admin sees all)
<code>/accounts/new</code>	AddAccountPage.jsx	Authenticated (User, Admin)
<code>/accounts/:id</code>	AccountDetailsPage.jsx	Authenticated (Owner, Admin)
<code>/transactions/deposit</code>	DepositPage.jsx	Authenticated (Owner, Admin)
<code>/transactions/withdraw</code>	WithdrawPage.jsx	Authenticated (Owner, Admin)
<code>/transactions/transfer</code>	TransferPage.jsx	Authenticated (Checking Account Owner, Admin)
<code>/transactions/history</code>	TransactionHistoryPage.jsx	Authenticated (User sees own; Admin sees global)
<code>/system/backup-restore</code>	BackupRestorePage.jsx	Admin Role Only
<code>/profile</code>	ProfilePage.jsx	Authenticated (Own Profile)

8 Domain Entities (Exact Fields)

8.1 Account

Field	Type	Database Constraints
AccountId	int (PK)	Identity, Clustered Primary Key
AccountNumber	nvarchar(20)	Unique Index (IX_Accounts_AccountNumber), Not Null
Balance	decimal(18,2)	Default: 0.00, Not Null
AccountType	AccountType (int)	0 = Savings, 1 = Checking, Not Null
IsActive	bit	Default: 1 (True), Not Null (Soft delete flag)
CreatedAt	datetime2	Not Null, UTC timestamp
UserId	nvarchar(450) (FK)	References AspNetUsers(Id), DeleteBehavior.Restrict, Not Null
RowVersion	byte[] (rowversion)	SQL Server rowversion column; EF Core concurrency token

8.2 Transaction

Field	Type	Database Constraints
TransactionId	int (PK)	Identity, Clustered Primary Key
AccountId	int (FK)	References Accounts(AccountId), DeleteBehavior.Restrict, Not Null
RelatedAccountId	int (FK, Nullable)	References Accounts(AccountId), DeleteBehavior.Restrict, Nullable
Amount	decimal(18,2)	Not Null, Greater than 0.00
TransactionType	TransactionType (int)	0 = Deposit, 1 = Withdrawal, 2 = Transfer, Not Null
TransactionMode	TransactionMode (int)	0 = Cash, 1 = Cheque, Not Null
Status	TransactionStatus (int)	0 = Pending, 1 = Completed, 2 = Failed, Not Null
ReferenceNumber	nvarchar(30)	Unique Index (IX_Transactions_ReferenceNumber), Not Null
TransactionDate	datetime2	Index (IX_Transactions_TransactionDate), Not Null
Description	nvarchar(500)	Nullable string

8.3 ApplicationUser / Identity Core

Entity	Fields & Key Properties
ApplicationUser	Id (nvarchar(450), PK), EmployeeCode (nvarchar(50), Unique Index), FirstName (nvarchar(100)), LastName (nvarchar(100)), Email (nvarchar(256), Unique), PasswordHash (nvarchar(max)), SecurityStamp, LockoutEnd, FailedAccessAttempts
IdentityRole	Id (nvarchar(450), PK), Name (nvarchar(256), Unique: Admin, User)
IdentityUserRole	UserId (FK), RoleId (FK) — Composite Primary Key

8.4 Domain Enumerations

public enum AccountType	{ Savings = 0, Checking = 1 }
public enum TransactionType	{ Deposit = 0, Withdrawal = 1, Transfer = 2 }
public enum TransactionMode	{ Cash = 0, Cheque = 1 }
public enum TransactionStatus	{ Pending = 0, Completed = 1, Failed = 2 }

9 Database Design — Indexes & Constraints (SQL Server)

Foreign keys enforce `ON DELETE RESTRICT / NO ACTION` to guarantee that banking ledgers remain immutable and cannot be orphaned. Concurrency is handled by SQL Server’s native `rowversion` data type.

```
-- Unique index on account numbers
CREATE UNIQUE NONCLUSTERED INDEX [IX_Accounts_AccountNumber]
ON [Accounts]([AccountNumber]);

-- Unique index on employee codes
CREATE UNIQUE NONCLUSTERED INDEX [IX_AspNetUsers_EmployeeCode]
ON [AspNetUsers]([EmployeeCode]);

-- Unique index on financial transaction reference numbers
CREATE UNIQUE NONCLUSTERED INDEX [IX_Transactions_ReferenceNumber]
ON [Transactions]([ReferenceNumber]);

-- Performance index for global chronological sorting
CREATE NONCLUSTERED INDEX [IX_Transactions_TransactionDate]
ON [Transactions]([TransactionDate]);

-- Composite index optimizing account-specific historical queries
CREATE NONCLUSTERED INDEX [IX_Transactions_AccountId_TransactionDate]
ON [Transactions]([AccountId], [TransactionDate]);
```

Design benefits of this schema:

- **Audit trail immutability:** Soft delete prevents physical deletion of accounts that have transaction rows attached.
- **Race condition prevention:** The `RowVersion` column increments on every SQL UPDATE, allowing EF Core to identify concurrent writes and throw `DbUpdateConcurrencyException`.
- **High-speed ledger pagination:** The composite index on `(AccountId, TransactionDate)` provides sub-millisecond query performance on paginated history lookups.

10 RBAC — Roles, Permissions & Enforcement

10.1 Roles

The system defines exactly two enterprise roles: `Admin` and `User`.

10.2 Seed Permissions by Role

Role	Permissions & Capabilities
Admin	Full system access: User management (CRUD), Role assignment, Global account inspection, Global transaction history querying, System save/restore (JSON backup & rehydration).
User	Self-service banking: Account opening, View owned accounts & balances, Deposit (Cash/Cheque), Withdraw (Cash/Cheque), Transfer between Checking accounts, View owned transaction history, Profile management.

10.3 Policy Registration (Program.cs)

```
// Role-based authorization policies applied to controller actions
[Authorize(Roles = "Admin")]
public class UsersController : ControllerBase { ... }

[Authorize(Roles = "Admin")]
public class SystemController : ControllerBase { ... }

// Granular resource ownership validated in services
var currentUserId = User.FindFirstValue(ClaimTypes.NameIdentifier);
var isAdmin = User.IsInRole("Admin");
if (account.UserId != currentUserId && !isAdmin)
    throw new UnauthorizedAccessException("Access denied to this account.");
```

10.4 Full Authorization Matrix

Capability	Admin	User (Owner)	User (Non-Owner)	Unauthenticated
Register / Login	Yes	Yes	Yes	Yes (Public)
View Account Balance	Yes	Yes	No (401/403)	No (401)
Open Account	Yes	Yes	Yes	No (401)
Deposit / Withdraw	Yes	Yes	No (401/403)	No (401)
Transfer Funds	Yes	Yes	No (401/403)	No (401)
Deactivate Account	Yes	Yes	No (401/403)	No (401)
System Save & Restore	Yes	No (403)	No (403)	No (401)
User Management (CRUD)	Yes	No (403)	No (403)	No (401)

11 Frontend ↔ Backend Permission Mapping

Feature / Action	Backend Endpoint	Required Role	Frontend UI Element / Behavior
Open Account	POST /api/v1/accounts	User / Admin	“Open Account” button & modal
Deactivate Account	DELETE /api/v1/accounts/{id}	Owner / Admin	“Deactivate” button on Account card
Deposit Funds	POST /api/v1/transactions/deposit	Owner / Admin	Deposit form submit button
Withdraw Funds	POST /api/v1/transactions/withdraw	Owner / Admin	Withdrawal form submit button
Transfer Funds	POST /api/v1/transactions/transfer	Owner / Admin	Transfer form submit button
System Backup	GET /api/v1/system/save	Admin	“Export Backup JSON” button (Admin only)
System Restore	POST /api/v1/system/restore	Admin	“Restore System” button & upload (Admin only)

12 Database Seed Data

12.1 Roles

Seeded idempotently on startup via `DbSeeder.cs`: `Admin` and `User`.

12.2 Default Administrator Account

The system seeds a default administrator account so the platform is operational upon initial deployment:

- **Email:** `admin@bank.com`
- **Employee Code:** `EMP001`
- **Role:** `Admin`
- **Password:** Configured securely through environment variables / secret store (e.g., `Admin@12345` in development environments).

12.3 Reference Data

Sample customer accounts (Savings and Checking) are provisioned during development to enable immediate verification of financial transfers, concurrency tokens, and cash-only rules.

13 Authentication — JWT Contract

13.1 Claims in the Access Token

```
{
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier": "usr_99182",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress": "customer@bank.com",
  "http://schemas.microsoft.com/ws/2008/06/identity/claims/role": "User",
  "employeeCode": "EMP042",
  "jti": "c89f2a0b-11d2-4e8c-8f43-098274aef12a",
  "iss": "BankAccountManagementSystem.API",
  "aud": "BankAccountManagementSystem.Client",
  "exp": 1735142400
}
```

- **Token Algorithm:** HMAC-SHA256 (`SecurityAlgorithms.HmacSha256`) with a 256-bit symmetric key.
- **Access Token TTL:** 60 minutes (configurable via `JwtSettings.ExpiryInMinutes`).
- **Clock Skew:** Set to `TimeSpan.Zero` for exact expiration enforcement.

13.2 Endpoints

Method	Route	Payload	Response
POST	<code>/api/v1/auth/register</code>	RegisterRequest (EmployeeCode, Names, Email, Password)	201 Created — RegisterResponse
POST	<code>/api/v1/auth/login</code>	LoginRequest (Email, Password)	200 OK — LoginResponse (Token, User Profile)
GET	<code>/api/v1/auth/me</code>	None (Requires Bearer Token)	200 OK — Authenticated User Profile

13.3 Password Policy & Lockout

- **Password Constraints:** Minimum 8 characters; must contain at least 1 digit, 1 lowercase letter, 1 uppercase letter, and 1 non-alphanumeric character.
- **Hashing:** PBKDF2 with HMAC-SHA256 and cryptographic per-user salt.
- **Lockout Policy:** 5 consecutive failed access attempts triggers a 5-minute account lockout.

13.4 Middleware Pipeline Order (Program.cs)

The middleware pipeline executes in strict sequential order:

1. CorrelationIdMiddleware	--> Assigns or propagates X-Correlation-ID
2. ExceptionMiddleware	--> Global catch: formats standard JSON error envelope
3. RequestLoggingMiddleware	--> Telemetry: logs method, path, status, duration (ms)
4. UseSwagger & SwaggerUI	--> Development API documentation
5. UseHttpsRedirection	--> Enforces HTTPS transport
6. UseStaticFiles	--> Serves static assets
7. UseRouting	--> Resolves endpoint routing
8. UseRateLimiter	--> Enforces fixed-window limiter on /auth/login
9. UseAuthentication	--> Validates JWT Bearer token
10. UseAuthorization	--> Enforces role and policy authorization
11. MapHealthChecks	--> GET /health endpoint
12. MapControllers	--> Dispatches request to controller action

14 Configuration & Environment Variables

14.1 Backend (appsettings.json)

Configuration Key	Purpose & Description
ConnectionStrings:DefaultConnection	SQL Server connection string (Server, Database, Security)
JwtSettings:SecretKey	256-bit symmetric encryption key used to sign JWTs
JwtSettings:Issuer	Valid issuer claim (BankAccountManagementSystem.API)
JwtSettings:Audience	Valid audience claim (BankAccountManagementSystem.Client)
JwtSettings:ExpiryInMinutes	Token validity period (e.g., 60 minutes)
Serilog:MinimumLevel	Logging threshold (Information in Dev, Warning in Prod)

14.2 Frontend (Vite Environment Variables)

Key	Description
VITE_API_BASE_URL	Base URL of the ASP.NET Core backend (e.g. http://localhost:5247/api/v1)

14.3 Configuration Files Required

Backend:	
appsettings.json	# Base configuration
appsettings.Development.json	# Local development overrides
Frontend:	
.env	# Local environment variables (VITE_API_BASE_URL)
.env.example	# Template with dummy values

15 API Contract (Representative Endpoints)

15.1 Authentication

Method	Route	Policy	Success Response
POST	/api/v1/auth/register	[AllowAnonymous]	201 Created — RegisterResponse
POST	/api/v1/auth/login	[AllowAnonymous], RateLimited	200 OK — LoginResponse (Token, User)
GET	/api/v1/auth/me	[Authorize]	200 OK — Current User Profile

15.2 Bank Accounts

Method	Route	Policy	Success Response
POST	/api/v1/accounts	[Authorize]	201 Created — Account Details
GET	/api/v1/accounts	[Authorize]	200 OK — List of Accounts
GET	/api/v1/accounts/{id}	[Authorize] (Owner/Admin)	200 OK — Account Details
GET	/api/v1/accounts/{id}/balance	[Authorize] (Owner/Admin)	200 OK — { accountId, balance }
DELETE	/api/v1/accounts/{id}	[Authorize] (Owner/Admin)	200 OK — Soft deactivation

15.3 Transactions

Method	Route	Policy	Success Response
POST	/api/v1/transactions/deposit	[Authorize] (Owner/Admin)	200 OK — TransactionResponse
POST	/api/v1/transactions/withdraw	[Authorize] (Owner/Admin)	200 OK — TransactionResponse
POST	/api/v1/transactions/transfer	[Authorize] (Owner/Admin)	200 OK — TransactionResponse
GET	/api/v1/transactions	[Authorize(Roles = "Admin")]	200 OK — PagedResult<TransactionResponse>
GET	/api/v1/transactions/account/{accountId}	[Authorize] (Owner/Admin)	200 OK — PagedResult<TransactionResponse>

15.4 System Save & Restore

Method	Route	Policy	Success Response
GET	/api/v1/system/save	[Authorize(Roles = "Admin")]	200 OK — SystemBackupDto (JSON file)
POST	/api/v1/system/restore	[Authorize(Roles = "Admin")]	200 OK — RestoreResponseDto

15.5 Error Responses (All Endpoints)

Status	Condition	Sample Response Body
400 Bad Request	Model validation failure / invalid inputs	<code>{ "success": false, "message": "Amount must be greater than zero." }</code>
401 Unauthorized	Missing or expired JWT Bearer token	Standard HTTP 401 Unauthorized
403 Forbidden	User attempts Admin endpoint (e.g. Save/Restore)	Standard HTTP 403 Forbidden
404 Not Found	Account or transaction ID does not exist	<code>{ "success": false, "message": "Account with ID 999 not found." }</code>
409 Conflict	Concurrent update on same account balance	<code>{ "success": false, "message": "The record was modified by another user." }</code>
422 Unprocessable	Insufficient funds / Savings cheque violation	<code>{ "success": false, "message": "Insufficient balance. Available: 500.00" }</code>
429 Too Many Req	Login attempts exceed 10/min from same IP	Standard HTTP 429 Too Many Requests
500 Internal Error	Unhandled server failure / database disconnect	<code>{ "success": false, "message": "Something went wrong.", "correlationId": "..." }</code>

16 API Request & Response Examples

POST /api/v1/auth/login

```
// Request
{ "email": "customer@bank.com", "password": "Password123!" }

// Response 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "usr_99182",
    "employeeCode": "EMP042",
    "firstName": "John",
    "lastName": "Doe",
    "email": "customer@bank.com",
    "roles": ["User"]
  }
}
```

POST /api/v1/accounts

```
// Request
{ "accountType": 1, "initialDeposit": 2500.00 }

// Response 201 Created
{
  "accountId": 104,
  "accountNumber": "ACC-20260928-8472",
  "balance": 2500.00,
  "accountType": "Checking",
  "isActive": true,
  "createdAt": "2026-09-28T09:15:00Z"
}
```

POST /api/v1/transactions/deposit (Cash on Savings Account)

```
// Request
{ "accountId": 101, "amount": 1000.00, "mode": 0 }

// Response 200 OK
{
  "transactionId": 501,
  "referenceNumber": "TXN-20260928-88392019",
  "amount": 1000.00,
  "transactionType": "Deposit",
  "transactionMode": "Cash",
  "status": "Completed",
  "transactionDate": "2026-09-28T09:20:00Z",
  "balanceAfter": 6000.00
}
```

POST /api/v1/transactions/deposit (Cheque on Savings Account — Rejected)

```
// Request
{ "accountId": 101, "amount": 1000.00, "mode": 1 }

// Response 422 Unprocessable Entity
{
  "success": false,
  "message": "Savings accounts allow cash transactions only. Cheque transactions are not permitted.",
  "statusCode": 422,
  "correlationId": "550e8400-e29b-41d4-a716-446655440000"
}
```

POST /api/v1/transactions/transfer (Checking to Checking)

```
// Request
{ "sourceAccountId": 104, "destinationAccountId": 105, "amount": 500.00, "mode": 1 }

// Response 200 OK
{
  "transactionId": 504,
  "referenceNumber": "TXN-20260928-99201122",
  "amount": 500.00,
  "transactionType": "Transfer",
  "transactionMode": "Cheque",
  "status": "Completed",
  "transactionDate": "2026-09-28T09:30:00Z",
  "balanceAfter": 2000.00
}
```

17 Repository / Service Interfaces & DTOs

17.1 Service Contracts

```
public interface IAccountService
{
    Task<AccountResponse> CreateAccountAsync(CreateAccountRequest request, string userId);
    Task<IEnumerable<AccountResponse>> GetAccountsAsync(string userId, bool isAdmin);
    Task<AccountResponse> GetAccountByIdAsync(int id, string userId, bool isAdmin);
    Task<decimal> GetBalanceAsync(int id, string userId, bool isAdmin);
    Task DeactivateAccountAsync(int id, string userId, bool isAdmin);
}

public interface ITransactionService
{
    Task<TransactionResponse> DepositAsync(DepositRequest request, string userId, bool isAdmin);
    Task<TransactionResponse> WithdrawAsync(WithdrawalRequest request, string userId, bool isAdmin);
    Task<TransactionResponse> TransferAsync(TransferRequest request, string userId, bool isAdmin);
    Task<PagedResult<TransactionResponse>> GetAccountTransactionsAsync(int accountId, TransactionHistoryQuery query,
string userId, bool isAdmin);
    Task<PagedResult<TransactionResponse>> GetAllTransactionsAsync(TransactionHistoryQuery query);
    Task<TransactionResponse> GetTransactionByIdAsync(int id, string userId, bool isAdmin);
}
```

17.2 Example Service Method (TransferAsync: ACID + Concurrency + Logging)

```

public async Task<TransactionResponse> TransferAsync(TransferRequest request, string userId, bool isAdmin)
{
    if (request.SourceAccountId == request.DestinationAccountId)
        throw new InvalidTransactionException("Source and destination accounts must be different.");

    using var dbTx = await _context.Database.BeginTransactionAsync();
    try
    {
        var source = await _context.Accounts.FirstOrDefaultAsync(a => a.AccountId == request.SourceAccountId)
            ?? throw new KeyNotFoundException("Source account not found.");
        var dest = await _context.Accounts.FirstOrDefaultAsync(a => a.AccountId == request.DestinationAccountId)
            ?? throw new KeyNotFoundException("Destination account not found.");

        // Validate active state
        if (!source.IsActive) throw new AccountInactiveException(source.AccountNumber);
        if (!dest.IsActive) throw new AccountInactiveException(dest.AccountNumber);

        // Validate ownership
        if (source.UserId != userId && !isAdmin)
            throw new UnauthorizedAccessException("Access denied to source account.");

        // Business Rule: Transfers only permitted between Checking accounts
        if (source.AccountType != AccountType.Checking || dest.AccountType != AccountType.Checking)
            throw new InvalidAccountTypeException("Transfers are only permitted between Checking accounts.");

        // Validate balance sufficiency
        if (source.Balance < request.Amount)
            throw new InsufficientBalanceException(source.Balance, request.Amount);

        source.Balance -= request.Amount;
        dest.Balance += request.Amount;

        var refNumber = GenerateReferenceNumber();

        var debitTx = new Transaction {
            AccountId = source.AccountId,
            RelatedAccountId = dest.AccountId,
            Amount = request.Amount,
            TransactionType = TransactionType.Transfer,
            TransactionMode = request.Mode,
            Status = TransactionStatus.Completed,
            ReferenceNumber = refNumber,
            TransactionDate = DateTime.UtcNow,
            Description = $"Transfer to {dest.AccountNumber}"
        };

        var creditTx = new Transaction {
            AccountId = dest.AccountId,
            RelatedAccountId = source.AccountId,
            Amount = request.Amount,
            TransactionType = TransactionType.Transfer,
            TransactionMode = request.Mode,
            Status = TransactionStatus.Completed,
            ReferenceNumber = refNumber,
            TransactionDate = DateTime.UtcNow,
            Description = $"Transfer from {source.AccountNumber}"
        };

        _context.Transactions.AddRange(debitTx, creditTx);
        await _context.SaveChangesAsync();
        await dbTx.CommitAsync();

        _logger.LogInformation("Transfer completed: {RefNumber}, Amount: {Amount}, From: {From}, To: {To}",
            refNumber, request.Amount, source.AccountNumber, dest.AccountNumber);

        return MapToResponse(debitTx, source.Balance);
    }
    catch (DbUpdateConcurrencyException ex)
    {
        await dbTx.RollbackAsync();
        _logger.LogWarning(ex, "Concurrency violation during transfer.");
    }
}

```

```
        throw;
    }
    catch
    {
        await dbTx.RollbackAsync();
        throw;
    }
}
```

17.3 ITransactionService

Orchestrates all debit, credit, and query operations. Ensures transactions are tracked, balances match exactly, and concurrency tokens are checked upon write.

17.4 Key DTOs

DTO Name	Purpose
DepositRequest / WithdrawalRequest	Validated inputs for cash/cheque deposits and withdrawals
TransferRequest	Validated input for inter-account fund transfers
TransactionResponse	Standard response representing transaction ledger entry
TransactionHistoryQuery	Pagination parameters (Page, PageSize) and date filters
SystemBackupDto	Root envelope for JSON system export & rehydration
RestoreResponseDto	Summary confirmation after system rehydration

18 Complete DTO Specification

```
DepositRequest:
  AccountId: int, required
  Amount: decimal, required, range [0.01, 10000000.00]
  Mode: TransactionMode (0 = Cash, 1 = Cheque), required

WithdrawalRequest:
  AccountId: int, required
  Amount: decimal, required, range [0.01, 10000000.00]
  Mode: TransactionMode (0 = Cash, 1 = Cheque), required

TransferRequest:
  SourceAccountId: int, required
  DestinationAccountId: int, required
  Amount: decimal, required, range [0.01, 10000000.00]
  Mode: TransactionMode (0 = Cash, 1 = Cheque), required

CreateAccountRequest:
  AccountType: AccountType (0 = Savings, 1 = Checking), required
  InitialDeposit: decimal, required, range [0.00, 10000000.00]

TransactionHistoryQuery:
  Page: int, default = 1, min = 1
  PageSize: int, default = 10, range [1, 100]
  StartDate: DateTime?, optional
  EndDate: DateTime?, optional
  TransactionType: TransactionType?, optional
```

19 Financial & Transaction Engine — Deep Detail

19.1 Balance & Transaction Formulas

```

Deposit:
    New Balance = Current Balance + Deposit Amount

Withdrawal:
    New Balance = Current Balance - Withdrawal Amount
    Constraint: Current Balance >= Withdrawal Amount (else throw InsufficientBalanceException)

Transfer:
    Source New Balance = Source Current Balance - Transfer Amount
    Destination New Balance = Destination Current Balance + Transfer Amount
    Constraint: Source Current Balance >= Transfer Amount
    Constraint: Source.AccountType == Checking AND Destination.AccountType == Checking
  
```

19.2 Transfer Processing Flow

```

Client POST /api/v1/transactions/transfer
|
v
Validate Request Model (Source != Destination, Amount > 0)
|
v
Begin EF Core Database Transaction (await _context.Database.BeginTransactionAsync())
|
+-- Fetch Source Account (Include RowVersion)
+-- Fetch Destination Account (Include RowVersion)
|
v
Check Active Status (Both accounts must have IsActive == true)
|
v
Check Ownership (Caller must own Source Account or hold Admin role)
|
v
Enforce Account Types (Both accounts must be Checking; Savings cannot transfer)
|
v
Verify Balance Sufficiency (Source Balance >= Amount)
|
v
Update Balances (Source -= Amount; Destination += Amount)
|
v
Create Transaction Ledgers (Debit leg with RelatedAccountId; Credit leg with RelatedAccountId)
|
v
Commit Changes (SaveChangesAsync -> CommitAsync)
| If DbUpdateConcurrencyException occurs -> RollbackAsync -> Throw Conflict (409)
| If any error occurs -> RollbackAsync -> Throw Exception
v
Return TransactionResponse
  
```

19.3 Concurrency Control & RowVersion Mechanics

Financial transactions are highly susceptible to lost updates if two requests read the same balance and update it concurrently. BAMS implements **Optimistic Concurrency Control** using SQL Server's `rowversion` column:

- `Account.RowVersion` is mapped in EF Core using `entity.Property(a => a.RowVersion).IsRowVersion()`.
- SQL Server automatically increments the `rowversion` value whenever an account record is updated.
- EF Core appends `WHERE AccountId = @id AND RowVersion = @original` to the SQL `UPDATE` statement.
- If another thread modified the balance first, zero rows are affected, causing EF Core to throw `DbUpdateConcurrencyException`.

- `ExceptionHandlerMiddleware` catches this exception and returns an HTTP 409 Conflict response, advising the client to retry.

20 Structured Logging & Audit Trail

20.1 Serilog Sinks

- **Console Sink:** Human-readable colored output for real-time development inspection.
- **File Sink:** Daily rolling log files saved to `Logs/bams-.log` with a 30-day retention limit and structured message templates.

20.2 Canonical Event Names

- `AUTH_LOGIN_SUCCESS`, `AUTH_LOGIN_FAILED`, `AUTH_LOGOUT`, `ACCOUNT_LOCKED`
- `ACCOUNT_CREATED`, `ACCOUNT_DEACTIVATED`
- `DEPOSIT_COMPLETED`, `WITHDRAWAL_COMPLETED`, `TRANSFER_COMPLETED`
- `CONCURRENCY_CONFLICT`, `INSUFFICIENT_FUNDS`, `SAVINGS_CHEQUE_VIOLATION`
- `SYSTEM_BACKUP_EXPORTED`, `SYSTEM_RESTORE_COMPLETED`
- `UNHANDLED_EXCEPTION`, `RATE_LIMIT_EXCEEDED`

20.3 Never Logged

Plaintext passwords, password hashes, JWT Bearer tokens, symmetric signing keys, and raw connection strings with credentials are strictly excluded from all log sinks.

20.4 Correlation ID Tracing

`CorrelationIdMiddleware` extracts `X-Correlation-ID` from incoming request headers or generates a new `Guid`. It attaches the ID to `HttpContext.Items["CorrelationId"]`, adds it to the HTTP response header, and passes it to `ExceptionHandlerMiddleware` and `RequestLoggingMiddleware` to ensure end-to-end request traceability across distributed logs.

21 SOLID Principles — Implementation Mapping

Principle	How BAMS Applies It	Concrete Implementation Location
S — Single Responsibility	Each class has exactly one reason to change. Controllers handle HTTP routing; services contain business rules; DbContext manages persistence; DTOs format payloads.	TransactionsController handles HTTP status codes; TransactionService handles balance arithmetic and ledger persistence.
O — Open/Closed	Systems are open for extension but closed for modification. New transaction types or backup formats can be introduced without breaking existing controllers.	ExceptionMiddleware uses C# pattern matching switch expressions to accommodate new domain exceptions without modifying existing catch blocks.
L — Liskov Substitution	Subtypes must be substitutable for their base types. ApplicationUser extends IdentityUser seamlessly.	ApplicationUser is passed to UserManager<ApplicationUser> without altering Identity behavior.
I — Interface Segregation	Clients depend only on focused, minimal interfaces rather than bloated abstractions.	IAccountService, ITransactionService, IAuthService, and ISystemBackupService are kept segregated.
D — Dependency Inversion	High-level business modules depend on abstractions (interfaces), never on concrete infrastructure classes.	Controllers depend on ITransactionService and IAccountService, injected via ASP.NET Core DI container in Program.cs.

22 Design Patterns — Implementation Mapping

Pattern	BAMS Usage	Architectural Purpose
Layered Architecture	Controller → Service → DbContext → SQL Server	Separation of presentation, business rules, and data access.
Service Layer Pattern	AccountService, TransactionService, AuthService	Encapsulates transaction business logic and domain rules.
Repository & Unit of Work	DbSet<T> (Repository) and DbContext (Unit of Work)	EF Core native implementation coordinating persistence and transaction boundaries.
DTO Pattern	DepositRequest, TransferRequest, TransactionResponse	Prevents over-posting, protects internal fields, and stabilizes API contracts.
Dependency Injection	builder.Services.AddScoped<TInterface, TImpl>()	Inversion of control; promotes testability and loose coupling.
Middleware (Chain of Responsibility)	CorrelationId → Exception → Logging → RateLimiter	Sequential request processing pipeline with short-circuiting capabilities.
Options Pattern	IOptions<JwtSettings>	Strongly typed configuration section binding from appsettings.json.

22.1 OOP Principles Applied

Concept	Concrete Application in BAMS
Encapsulation	Banking models enforce private state where necessary and validate data via properties; DTOs encapsulate request payloads.
Abstraction	Controllers consume interfaces (ITransactionService) without knowledge of SQL Server or EF Core internals.
Inheritance	ApplicationUser inherits from IdentityUser; custom domain exceptions (InsufficientBalanceException) inherit from InvalidOperationException.
Polymorphism	ExceptionMiddleware polymorphically handles distinct exception types through type-pattern matching.

23 Security, Exception Handling & Production Hardening

23.1 Exception Handling Contract

`ExceptionHandlerMiddleware` catches all downstream exceptions and translates them into uniform, sanitized JSON responses:

```
public async Task InvokeAsync(HttpContext context)
{
    try { await _next(context); }
    catch (Exception ex) { await HandleExceptionAsync(context, ex); }
}

private async Task HandleExceptionAsync(HttpContext context, Exception ex)
{
    var statusCode = ex switch
    {
        DbUpdateConcurrencyException => 409,
        UnauthorizedAccessException => 401,
        KeyNotFoundException          => 404,
        ArgumentException              => 400,
        InvalidOperationException        => 422, // InsufficientBalance, AccountInactive, InvalidAccountType
        _                             => 500
    };

    var message = _env.IsDevelopment() ? ex.Message : "Something went wrong. Please try again later.";
    // Returns { success = false, message, statusCode, correlationId }
}
```

23.2 Security Design

- **Authentication:** Signed JWT tokens containing user claims, role claims, and 60-minute expiration.
- **Brute-Force Protection:** Fixed-window rate limiting on login restricts requests to 10/min per remote IP.
- **SQL Injection Defense:** EF Core parameterizes all queries; raw concatenated SQL is prohibited.
- **Account Ownership Validation:** Normal users are strictly restricted to inspecting and transferring from accounts matching their own `ClaimTypes.NameIdentifier`.

23.3 Security Testing Checklist

Expired JWT • Invalid signature • Missing token • Wrong role • Non-owner account access • SQL injection strings • Negative deposit amounts • Brute-force rate limit trigger • Secret leak audit in logs.

23.4 Deployment

The backend compiles into a self-contained or framework-dependent ASP.NET Core application hosted via Kestrel behind IIS, Nginx, or Windows Services. The React SPA is compiled using `npm run build` into static assets served via CDN or web server.

23.5 Exact Configuration / Launch Profile

```
{
  "profiles": {
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7190;http://localhost:5247",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

23.6 Git / Branch / Commit Rules

- **Branch Strategy:** `main` (production ready) ← `develop` ← `feature/<phase-name>` .
- **Commit Convention:** Conventional Commits: `feat(transactions): implement atomic cheque transfer` .
- **Secrets:** Never commit passwords, signing keys, or connection strings.

24 Testing Matrix

Module	Unit Tests	Integration Tests
Authentication	Password hashing, JWT generation, claim parsing, invalid login rejection	POST <code>/api/v1/auth/login</code> happy path, wrong password → 401, rate limiter → 429
Accounts	Account number generation, initial deposit validation, soft deactivation flag	POST <code>/api/v1/accounts</code> as User → 201; non-owner access → 401/403
Transactions	Savings cash-only rule, Checking cash & cheque support, balance arithmetic	Cash deposit → 200; Cheque on Savings → 422; Insufficient balance → 422
Transfers	Transfer between Checking accounts, debit & credit verification	Atomic transfer → dual ledger creation; Concurrency collision → 409 Conflict
System Save/Restore	JSON serialization schema, entity rehydration, <code>AsNoTracking</code> query	GET <code>/system/save</code> as User → 403; as Admin → 200; Restore invalid JSON → 400

25 Sample Test Case Table

Test ID	Module	Scenario	Expected Result
TC001	Auth	Valid login credentials	200 OK — Signed JWT token generated
TC002	Auth	Invalid password	401 Unauthorized
TC003	Auth	Login attempts exceed 10/min from same IP	429 Too Many Requests
TC004	Accounts	Create Savings account with initial deposit	201 Created — AccountNumber generated
TC005	Accounts	User accesses another customer's account balance	401 Unauthorized / 403 Forbidden
TC006	Transactions	Deposit cash into Savings account	200 OK — Balance credited
TC007	Transactions	Deposit cheque into Savings account	422 Unprocessable Entity (Cash only rule)
TC008	Transactions	Deposit cheque into Checking account	200 OK — Balance credited
TC009	Transactions	Withdrawal exceeding current balance	422 Unprocessable Entity (Insufficient funds)
TC010	Transactions	Transfer funds from Savings account	422 Unprocessable Entity (Savings transfer prohibited)
TC011	Transactions	Transfer funds between Checking accounts	200 OK — Dual transaction records written
TC012	Transactions	Transaction on soft-deactivated account	422 Unprocessable Entity (Account inactive)
TC013	Concurrency	Simultaneous balance update race condition	409 Conflict (RowVersion mismatch caught)
TC014	System	Regular user accesses GET /api/v1/system/save	403 Forbidden
TC015	System	Admin exports and rehydrates system backup	200 OK — Full database restored atomically

26 Required Diagrams

The following technical diagrams document the structural and dynamic aspects of the system:

- **High-Level System Architecture:** Decoupled React 18 SPA + ASP.NET Core 8 Web API + SQL Server.
- **Component & Middleware Pipeline:** Correlation ID → Exception → Logging → RateLimiter → JWT Auth → Controllers.
- **Entity-Relationship Schema:** AspNetUsers, AspNetRoles, Accounts (with RowVersion), Transactions (Dual FKs).
- **Authentication Sequence:** Client → RateLimiter → Identity → JWT token generation → Axios storage.
- **Atomic Transfer Sequence:** BeginTransactionAsync → Balance checks → Concurrency token check → CommitAsync.
- **Save & Restore Flow:** JSON serialization → Database truncation → Atomic transactional rehydration.

27 Final Acceptance Criteria

- ☐ Application builds with zero compilation errors and zero warnings.
- ☐ SQL Server migrations apply cleanly on a fresh database.
- ☐ User registration, login, and JWT Bearer authentication function correctly.
- ☐ Fixed-window rate limiting prevents brute-force attacks on the login endpoint (429 returned).
- ☐ Role-based authorization strictly isolates Admin and User capabilities.
- ☐ Account creation generates unique account numbers formatted as ACC-YYYYMMDD-XXXX.
- ☐ Savings accounts strictly permit cash transactions and reject cheque deposits/transfers (422 returned).
- ☐ Checking accounts support both cash and cheque transactions and inter-account transfers.
- ☐ Withdrawals and transfers exceeding available funds throw InsufficientBalanceException (422 returned).
- ☐ Transfers execute atomically under IDbContextTransaction, writing debit and credit legs.
- ☐ Optimistic concurrency via RowVersion detects balance update collisions (409 returned).

- ☐ Account deactivation is soft (IsActive = false), preserving transaction history foreign keys.
- ☐ Transactions on deactivated accounts are rejected with AccountInactiveException (422 returned).
- ☐ Read-only queries use AsNoTracking() for optimal query performance.
- ☐ Transaction history supports pagination (Skip/Take) and date-range filtering.
- ☐ Admin save/restore exports complete database to JSON and rehydrates it atomically.
- ☐ ExceptionMiddleware sanitizes unhandled errors, omitting stack traces and SQL details.
- ☐ Serilog records structured logs to Console and daily rolling files under Logs/bams-.log.
- ☐ CorrelationIdMiddleware tracks requests across logs and responses via X-Correlation-ID.
- ☐ GET /health probes database connectivity, returning 200 OK or 503 Service Unavailable.

28 Definition of Done (Per Module)

An AI coding tool or developer must not report a module as “complete” until every item below is verified:

- [] Entity and enums created with appropriate data types (decimal(18,2), byte[] RowVersion).
- [] EF Core Fluent API configuration completed with indexes and DeleteBehavior.Restrict.
- [] Database migration generated and applied cleanly against SQL Server.
- [] Scoped service interface and implementation created with business rules.
- [] Request and response DTOs created with Data Annotation validation attributes.
- [] Controller action implemented, returning ActionResult<T> with appropriate HTTP status codes.
- [] Authorization attributes ([Authorize], [Authorize(Roles = "Admin")]) applied.
- [] Structured logging added for domain events via ILogger.
- [] Endpoints manually verified via Swagger UI.
- [] Frontend API client service functions created.
- [] React page components created with loading, error, and empty state handlers.
- [] Frontend form validation matches backend Data Annotation constraints.
- [] Backend and frontend compile with zero errors.

29 Build Order & Working Agreement for an AI Coding Tool

29.1 Build Order (Waterfall-Sequenced)

1. **Data Layer & Identity Core:** ApplicationUser, ApplicationDbContext, DbInitializer, InitialIdentity migration.
2. **Authentication & JWT:** JwtSettings, AuthService, AuthController, RateLimiter configuration, Swagger Bearer setup.
3. **Accounts Domain:** Account entity, AddBankAccount migration, AccountService, AccountsController.
4. **Transactions Domain:** Transaction entity, AddTransactions migration, RowVersion concurrency token, TransactionService (Deposit, Withdraw, Transfer).
5. **Transaction History & Querying:** AddTransactionHistoryIndexes migration, paginated LINQ queries, query DTOs.
6. **System Backup & Rehydration:** SystemBackupDto, SystemBackupService, SystemController (GET /save, POST /restore).
7. **Production Hardening:** Serilog configuration, CorrelationIdMiddleware, ExceptionMiddleware, RequestLoggingMiddleware, HealthChecks.
8. **React Frontend:** Vite setup, Axios interceptors, AuthContext, ProtectedRoute, Dashboard, Accounts, Transactions, Backup/Restore pages.
9. **End-to-End Verification:** Comprehensive integration testing across the full banking lifecycle.

29.2 Working Agreement

If this document is handed to an AI coding tool to generate or maintain the code, it must follow these rules:

- **Build incrementally, phase by phase (§29.1)** — never attempt to generate the entire application in one pass.
- **Before modifying an existing file, read its current contents and reuse existing models rather than redefining them.**
- **Never create duplicate or conflicting classes for the same concept.**
- **After each phase: compile the affected projects and fix errors before proceeding.**
- **Never claim a feature is implemented unless it actually exists and compiles in the code.**
- **For each non-trivial architectural decision, state: what was chosen, why, where it's implemented, and the trade-off.**
- **Do not introduce a design pattern, technology, or module beyond what is specified in this document.**

30 One-Minute Team Explanation

“The Bank Account Management System (BAMS) is a decoupled React 18 SPA and ASP.NET Core 8 Web API application using Entity Framework Core 8 and Microsoft SQL Server. It follows a Clean Layered Architecture across API, Middleware, Service, and Data layers. Dependency injection, the DTO pattern, and EF Core as the Unit of Work provide strong separation of concerns. Security is established through ASP.NET Core Identity with PBKDF2 password hashing, signed JWT Bearer tokens, rate limiting on authentication, and account ownership enforcement. Business logic strictly distinguishes Savings accounts (cash transactions only) from Checking accounts (cash and cheque transactions, inter-account transfers). Financial transfers execute atomically under explicit database transactions, while account balance race conditions are eliminated using SQL Server rowversion optimistic concurrency tokens. Operational visibility is achieved via Serilog rolling file logs, X-Correlation-ID request tracing, and database health checks. The platform is designed and delivered via a single Waterfall SDLC pass — requirements and low-level design fixed up-front in this document, verified through systematic integration testing.”

31 AI Project Generation Prompt

This is the ready-to-copy prompt to paste into an AI coding tool alongside this document:

You are the senior software architect and full-stack developer responsible for implementing the Bank Account Management System (BAMS).

Read this entire document before writing any code.

This document is the SINGLE SOURCE OF TRUTH. Implement the complete project according to every requirement specified in it.

Technology:

- ASP.NET Core Web API, C# (.NET 8.0 LTS)
- React 18 + Vite (JavaScript / JSX)
- Microsoft SQL Server
- Entity Framework Core 8.0.10 (Code-First)
- ASP.NET Core Identity
- JWT Bearer Authentication (HMAC-SHA256)
- Serilog (Console + Rolling Daily File Sink in Logs/bams-.log)
- ASP.NET Core RateLimiter (Fixed Window on /api/v1/auth/login)
- HealthChecks (/health)
- Swagger / OpenAPI

Do not:

- change the architecture, database (use SQL Server, NOT PostgreSQL), or technology stack
- remove or rename modules, entities, roles, or permissions
- invent endpoints, DTO fields, or permissions not in this document
- bypass authorization on any endpoint
- expose EF Core entities directly through the API
- put business logic inside controllers or React components
- create placeholder implementations (TODO, NotImplementedException, empty methods)
- skip database migrations
- commit secrets, .env files, or generated build output

Follow the Build Order (§29.1) and AI Coding Execution Contract (§2) sections of this document exactly.

After each phase:

1. Build (backend and frontend)
2. Run tests and verify zero compiler errors
3. Verify migrations apply cleanly to SQL Server
4. Continue only when the phase is green

Before declaring any module complete, verify it against the Definition of Done checklist in this document.

At the end of the project:

- run all backend verification tests
- build the frontend
- verify authorization for every role/route combination in the authorization matrix
- verify every item in the Final Acceptance Criteria section

Do not claim the project is complete until every acceptance criterion in this document has been satisfied and verified, not assumed.

End of Document. This LLD is the complete, consolidated implementation contract for the BAMS – nothing outside it should be assumed necessary, and nothing inside it is optional.